TRIALMON: SAS/R macro library for unattended trial-monitoring

A deterministic, **no-AI** macro library that turns your already-validated monitoring programs into a scheduled, self-escalating loop: **ingest → freshness gate → checks → roll-up → digest**, plus a **tier-2 urgent alert** on any de-duplicated RED finding, always closed by a **heartbeat**. Built for the "covering while on leave" use case, but it's the standing monitoring backbone for any study.

> ***What it is, and isn't.*** *These macros* ***detect, package, and email*** *pre-specified deterministic flags. They* ***do not triage, interpret, or adjudicate.*** *A flag is a prompt for a human; the medical monitor and the covering biostatistician make every call. Validate the threshold logic + scheduling wrapper as study programs; keep anything reported independently double-programmed.*

Files

File	What
`tm_macros.sas`	The library: all `%tm_*` macros (written for Base SAS 9.4M8 or later; not executed, no SAS license; no third-party packages).
`monitor_driver.sas`	The daily job: orchestrates ingest → freshness → guard → checks → digest → alert → heartbeat. Schedule in batch under a **service account**.
`tm_config.sas`	**Per-study constants** (paths, recipients, the medical monitor, feeds, thresholds): `%include` before the driver, so a second study or a rotating MM is one file, not a code edit.
`tm_watchdog.sh`	The **independent dead-man's switch**: schedule on a *different* host/account; reads the heartbeat and screams if the job didn't run, ran late, or ran on stale data.
`tm_companion.R`	R companion (haven / dplyr / gt / blastula) for R-native shops; schedule with `cronR` / `taskscheduleR`, pin with `renv`.

Quick start (SAS)

```sas
%include "/opt/trialmon/tm_macros.sas";
%tm_init(study=CP101, root=/opt/trialmon/cp101, smtp=smtp.internal.example.com,
statelib=/opt/trialmon/cp101/state);
%tm_freshness(feeds=adam.adlb adam.adeg adam.adae, maxage=26);   /* GATE first */
%tm_chk_hyslaw(in=adam.adlb_edish);                              /* deterministic flag */
%tm_status(in=_tm_hyslaw);                                       /* roll up severity */
%tm_digest(title=CP-101 Daily Safety Digest, sections=_tm_hyslaw, to=&support);
%tm_alert(in=_tm_red_new, to=&support, backup=&backup, alias=&alias,
mm_name=%str(Dr. Okafor), mm_phone=+1-555-0100, evidence=&packet);
%tm_heartbeat(records=&n, watcher_to=&backup);
```

Schedule it (service account, **not** a personal login):

```bat
sas.exe -sysin monitor_driver.sas -log "logs\cp101_%date%.log" -batch -noterminal
:: Windows Task Scheduler
schtasks /create /tn "CP101_monitor" /tr "run_monitor.bat" /sc DAILY /st 06:00 /ru svc_trialmon
```

```cron
# Linux cron
0 6  1-5  /opt/jobs/run_monitor.sh   # calls sas -sysin monitor_driver.sas -batch
```

Macro catalog

Core / orchestration

Macro	Purpose	Key parameters
`%tm_init`	Initialize a run: paths, options, run id, SMTP, persistent **state** library.	`study= root= smtp= statelib=`
`%tm_latest`	Resolve the **newest dated** file in a directory (unambiguous "latest export").	`dir= ext= outvar=`
`%tm_status`	Roll a flagged dataset's worst `sev` (GREEN<AMBER<RED) into the run status.	`in= sevvar=`

Fail-loud & the freshness gate (run first)

| `%tm_assert` | **Fail loud:** on a false condition, set the return code, force status to ERROR, email the backup, write a FAILED heartbeat, optionally abort. Wrap every fragile step. | `cond= msg= abort=` |

| `%tm_guard` | Before any safety threshold: assert the expected columns exist, are numeric, and pass a range-sanity check (catches a unit change / renamed var). | `ds= numvars= sanity=` |

| `%tm_freshness` | Compare each feed's newest timestamp to now; a **missing/dead feed flags loudest**. **Stale-green is worse than red.** Writes one row per feed. | `feeds= tsvar= maxage= out=` |

| `%tm_evidence` | Build the per-RED-participant evidence packet (eDISH scatter with 3×/2× reference lines + lab trajectory) as a one-page PDF the alert attaches. | `in= lab= out=` |

Safety checks *(each → a flagged dataset with a `sev` column)*

| `%tm_chk_hyslaw` | eDISH / Hy's-Law screening: ALT or AST >×ULN **and** TBili >×ULN. *Screening flag, not a diagnosis.* | `altmult=3 astmult=3 bilmult=2 alpmult=2` |

| `%tm_chk_qtcf` | QTcF absolute / change-from-baseline tiers (ICH E14). | `abs_red=500 dqt_red=60 abs_amb=480 dqt_amb=30` |

| `%tm_chk_ae` | AE/SAE running tally by cohort: **participant-incidence and event counts**. | `in= out=` |

| `%tm_chk_dlt` | Candidate-DLT tally vs the 3+3 rule (against an **adjudicated** DLT flag + completed window). *Does not make the escalation decision.* | `in= out=` |

| `%tm_chk_labs` | Out-of-range / PCS / shift flags. | `in= out=` |

Operational checks

| `%tm_chk_enroll` | Enrollment vs plan KRI. | `kri=0.80` |

| `%tm_chk_queries` | Open-query aging. | `age_amb=30` |

| `%tm_chk_visits` | Overdue / out-of-window visits (uses "today"). | `in= out=` |

| `%tm_chk_ixrs` | Randomization/IXRS reconciliation: **discrepancies escalate to a human**, never auto-resolved. | `clin= ixrs=` |

Notification & safeguards

| `%tm_dedup` | Alert each distinct finding **once** (persistent seen-flags store), no alert storm. | `in= key= out=` |

| `%tm_email` | `FILENAME EMAIL` wrapper; de-identified body, record-level detail attached. | `to= cc= subject= bodyfile= attach= importance=` |

| `%tm_digest` | Tier-1 ODS digest with the RAG status; always appends a **heartbeat** line. | `title= sections= to= dest=` |

| `%tm_alert` | Tier-2 urgent alert: severity routing + **"CONTACT THE MEDICAL MONITOR"** + evidence packet. | `in= to= backup= alias= mm_name= mm_phone= evidence=` |

| `%tm_heartbeat` | Dead-man's switch: append a positive "ran OK" ping; an **independent watcher** escalates if it stops. | `records= watcher_to=` |

The four safeguards (why it won't fail silently)

1. **Heartbeat / dead-man's switch** (`%tm_heartbeat` + an *independent* watcher on a second host): a dead job announces itself.
2. **Data-freshness gate, run first** (`%tm_freshness`): a frozen feed is caught before any "green" is trusted.
3. **De-duplication** (`%tm_dedup`): one real event alerts once.
4. **Primary + backup + role alias** (`%tm_alert` recipients) with an acknowledgment SLA: a 6 a.m. alert never lands in a void.

Rule: "no news" is **never** "all clear." A successful, data-fresh run always emits an explicit GREEN digest with a heartbeat; the *absence* of any message is itself escalated.

Validation & scope

- **Reuses validated computation.** The macros surface outputs from the same validated programs already qualified for the study: they automate the *delivery/cadence*, not the method. The thin scheduling wrapper + threshold logic are validated like any study program (spec → independent review → documented test → change control).

- **Classify every output:** `REPORTABLE` (must be QC'd / double-programmed before it ships) vs `SIGNAL-ONLY` (single-program, human-confirmed). The monitor's emails are SIGNAL-ONLY.
- **No PHI in email bodies**: de-identified signal only; record-level detail stays in the attached report on the validated share.
- **Honest limit.** Deterministic checks only, no triage, narrative, or judgment.

Change log: v3 (October 2026)

- **Hy's-Law aminotransferase limb:** now >= 3×ULN, as FDA's DILI guidance (2009) words it ("3-fold or greater"); total bilirubin stays > 2×ULN. The SAS macro, its config comment and the R companion changed together.

Change log: v2 (corrected after a review of the SAS and clinical logic)

- **Hy's-Law (clinical, fixed first).** The RED screening flag (ALT or AST >3×ULN **and** total bilirubin >2×ULN) is now **strict `>`** and is **never downgraded by a high ALP** (a raised ALP does **not** exclude Hy's Law). The R-ratio (ALT/ULN ÷ ALP/ULN) is reported as *context only* (hepatocellular when R≥5). The prior code wrongly demoted a true case to AMBER on ALP≥2×: the single most dangerous bug, removed.
- **Freshness gate rewritten** as one coherent per-feed pass that actually writes results; a **missing/dead feed flags** (never silently passes green); no leaked dataset handles.
- **Digest fixed**: the heartbeat line, the `importance` (resolved with macro `%if`, not run-time `IFC`), and the digest **body file** are all created/correct now.
- **De-dup keyed by participant** (`%tm_dedup` signature includes USUBJID/dose) so a *second* participant's RED finding is never suppressed as a duplicate.
- **AETOXGR** compared via `input(AETOXGR,best.)` (it's character in CDISC); SAE routes to AMBER by default (scope RED to SUSAR/related if preferred). QTcF adds the 450 ms watch tier.
- **Fail-loud added** (`%tm_assert` / `%tm_guard`), an **evidence-packet builder** (`%tm_evidence`), a **config file**, and an **independent watchdog**: the gaps the review flagged for an unattended job.
- **R companion**: strict Hy's-Law, NA-safe freshness gate, `%.0f` (not `%d`) formatting, a `tm_read()` that lowercases CDISC column names, and the seen-store committed **only after** a successful send.

Companion to the SAS/R Trial-Monitoring Automation wiki and the "Coverage while on leave" worked example. Column names (ALT_X, QTCF, DLTFL, etc.) are placeholders: map to your validated ADaM/SDTM specs. Illustrative; adapt LIBNAMEs, paths, SMTP, and thresholds to your protocol. The logic was reviewed but has not been executed in SAS (no license); validate it as a study program before unattended use.